

1. Gestion des Membres Statiques (Compteur)

- **Déclaration** : Un membre statique (ex: static int compteur;) doit être déclaré en private.
- **Initialisation** : L'initialisation **doit** se faire à l'extérieur de la classe, dans le fichier source, sous la forme : int NomClasse::compteur = 0;.
- **Logique** : Le compteur doit être incrémenté dans le constructeur (++compteur) pour garantir un identifiant unique à chaque objet.

2. Allocation Dynamique et Gestion Mémoire

- **Pointeurs** : Pour un tableau de poids ou de montants, on utilise un pointeur (ex: float* tab;).
- **Allocation** : L'allocation se fait dans le constructeur avec l'opérateur new : tab = new float[taille];.
- **Initialisation** : Après l'allocation, une boucle for est obligatoire pour initialiser les valeurs (souvent à 0).
- **Destructeur** : Pour éviter les fuites mémoire (memory leaks), l'utilisation de delete [] tab; est impérative si l'allocation a été faite avec new[].

3. Le Constructeur de Recopie (Deep Copy)

- **Principe** : Le constructeur de recopie doit effectuer une **copie profonde**.
- **Erreur Critique** : Faire this->tab = source.tab; est une erreur grave (Shallow Copy). Cela copie l'adresse mémoire et non les données.
- **Procédure Correcte** :
 1. Allouer un nouveau tableau de la même taille : this->tab = new float[source.taille];.
 2. Copier les valeurs une par une via une boucle : for(int i=0; i<n; i++) { this->tab[i] = source.tab[i]; }.

4. Fonctions Amies (Friend Functions)

- **Déclaration** : À l'intérieur de la classe avec le mot-clé friend (ex: friend void afficher(const NomClasse &obj);).
- **Définition** : À l'extérieur de la classe, **sans** utiliser NomClasse::.
- **Accès** : La fonction amie accède aux membres privés en utilisant l'opérateur point sur l'objet passé en paramètre (obj.attribut_privé).